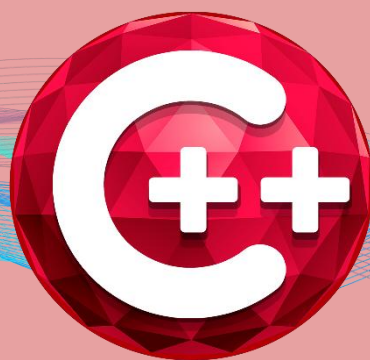


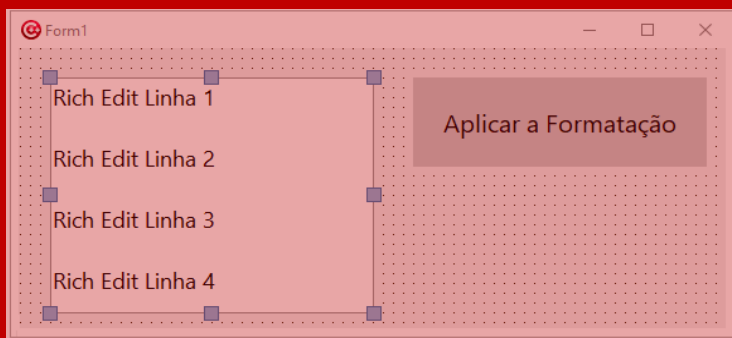


RAD Studio | C++Builder 13

FLORENCE



MÓDULO 1 - BÁSICO



COMPONENTE
ENTRADA
MANIPULAÇÃO
DADOS
C++ BUILDER



Criado por: Thiago Santos

MVP Embarcadero. 2026

SUMÁRIO

Sumário	2
Introdução ao Curso de C++Builder 13 Florence	3
O que você vai aprender neste curso.....	4
Sobre o instrutor.....	5
Componentes para entrada e manipulação de dados	7
Edit.....	7
MaskEdit.....	8
DateTimePicker	10
SpinEdit	11
Memo.....	11
RichEdit	13
ComboBox	15
CheckBox	16
RadioButton	17
RadioGroup.....	17
Panel.....	19
GroupBox	20
Bevel.....	21

Seja bem-vindo!!

Introdução ao Curso de C++Builder 13 Florence

Este curso foi desenvolvido para apresentar o **C++Builder 13 Florence**, a versão mais recente da plataforma de desenvolvimento da Embarcadero, com foco em quem deseja aprender **C++ de forma prática, produtiva e aplicada ao desenvolvimento de aplicações reais**.

Ao longo desta formação, o aluno será introduzido aos principais conceitos do **C++ funcional aplicado ao C++Builder**, entendendo não apenas a linguagem, mas também como utilizá-la de forma eficiente dentro de um ambiente RAD (Rapid Application Development), que acelera o processo de criação de software sem *abrir mão de desempenho e controle*.

A importância de utilizar o C++Builder 13 Florence

Utilizar a versão mais recente do C++Builder é fundamental para garantir **compatibilidade com tecnologias atuais**, maior estabilidade da IDE e acesso aos recursos mais modernos da linguagem C++. O C++Builder 13 Florence traz melhorias significativas no compilador baseado em Clang, suporte aprimorado ao padrão moderno do C++, além de uma IDE mais rápida, segura e alinhada às necessidades do mercado atual.

Aprender com uma versão atual evita retrabalho futuro, reduz problemas de compatibilidade e prepara o aluno para atuar em projetos profissionais, corporativos e comerciais, utilizando ferramentas reconhecidas e amplamente adotadas por empresas.

O que você vai aprender neste curso

Este curso tem como objetivo fornecer uma **base funcional e prática**, permitindo que o aluno desenvolva aplicações reais desde os primeiros módulos. Entre os principais conhecimentos adquiridos, destacam-se:

- Fundamentos do C++ aplicados ao C++Builder
- Estrutura da IDE e fluxo de desenvolvimento
- Criação de aplicações visuais utilizando VCL
- Manipulação de eventos e componentes
- Lógica de programação aplicada a projetos reais
- Integração básica com banco de dados
- Boas práticas para desenvolvimento em C++Builder

Ao final do curso, o aluno terá conhecimento suficiente para **criar aplicações funcionais**, entender projetos existentes e evoluir para níveis mais avançados da linguagem e da ferramenta.

Por que escolher o C++Builder

O C++Builder se diferencia por unir o **alto desempenho do C++** com a **produtividade do desenvolvimento visual**. Ele permite criar aplicações robustas, rápidas e profissionais com muito menos código do que abordagens tradicionais, mantendo total controle sobre a linguagem e o desempenho.

Além disso, o C++Builder é amplamente utilizado em sistemas corporativos, aplicações industriais, softwares comerciais e soluções que exigem **performance, estabilidade e longevidade**. Aprender C++Builder é investir em uma tecnologia madura, sólida e ainda extremamente relevante no mercado.

Este curso é o primeiro passo para quem deseja dominar o **C++ moderno aplicado ao desenvolvimento profissional**, utilizando uma das ferramentas mais completas e produtivas disponíveis atualmente.

Está pronto? Vamos começar!!

Sobre o instrutor

Meu nome é **Thiago Santos** e sou profissional da área de tecnologia, atuando há vários anos com **desenvolvimento de software, análise de negócios e treinamentos técnicos**, com foco na linguagem **C++** e na plataforma **C++Builder**.

Ao longo da minha trajetória profissional, tive a oportunidade de trabalhar diretamente com projetos reais, ambientes corporativos e soluções que exigem **alto desempenho, estabilidade e manutenção a longo prazo**. Essa experiência prática me permitiu entender não apenas a linguagem C++, mas também como aplicá-la de forma eficiente no dia a dia de empresas e equipes de desenvolvimento.

Sou **MVP da Embarcadero na linguagem C++**, reconhecimento concedido a profissionais que contribuem ativamente para a comunidade, compartilham conhecimento e utilizam as tecnologias Embarcadero em projetos reais. Além disso, atuo como **instrutor, agente de negócios e consultor técnico**, auxiliando empresas e desenvolvedores na adoção e evolução do C++Builder em seus sistemas.

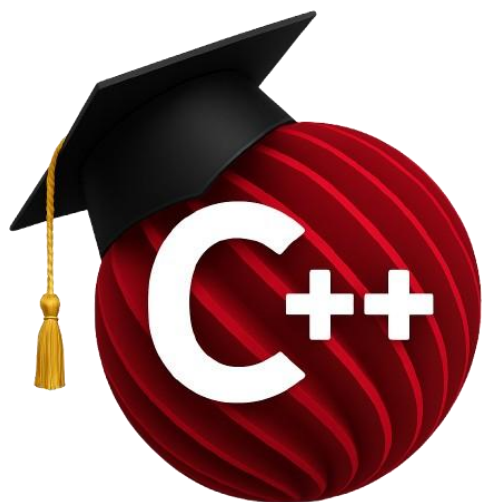
Neste curso, meu objetivo não é apenas ensinar comandos ou conceitos isolados, mas **transmitir a forma correta de pensar, estruturar e desenvolver aplicações em C++Builder**, com base em experiências reais de mercado. Todo o conteúdo foi planejado para ser direto, prático e aplicável, evitando excesso de teoria desconectada da prática.

Acredito que aprender programação vai muito além de escrever código: envolve entender processos, tomar decisões técnicas corretas e criar soluções que realmente funcionem. É exatamente essa visão que compartilho ao longo deste curso.

Seja bem-vindo(a) à formação. Espero que este conteúdo contribua de forma significativa para sua evolução profissional e técnica no desenvolvimento com **C++Builder**.

Meus contatos e plataformas estão disponíveis:

<https://sam-cpp.wordpress.com/>



COMPONENTES PARA ENTRADA E MANIPULAÇÃO DE DADOS

PERFIL

Este treinamento acadêmico visa apresentar ao aluno um ambiente integrado de desenvolvimento de alta produtividade, que permita a criação de aplicações de todos os tipos e para as mais importantes plataformas da atualidade.

Neste capítulo serão apresentados os conceitos e componentes envolvidos no acesso a dados e componentes de manipulação dos mesmos.

Caso queira se aprofundar e se especializar no conhecimento da ferramenta, sua linguagem, tecnologias e recursos, uma lista de centros de treinamentos oficiais está disponível:

www.embarcadero.com.br

CONTATO

academico@embarcadero.com.br

O que você verá:

Este módulo do treinamento acadêmico abordará os principais componentes para entrada e manipulação de informação, como:

- Edit
- MaskEdit
- DateTimePicker
- SpinEdit
- Memo
- RichEdit
- ComboBox
- CheckBox
- RadioButton
- RadioGroup
- Panel
- GroupBox
- Bevel

HABILIDADES

Componentes para entrada e manipulação de dados

Para facilitar o desenvolvimento de **aplicações visuais** e melhorar a **usabilidade do usuário final**, o **C++Builder** disponibiliza um amplo conjunto de componentes que permitem a interação do usuário com a aplicação. Esses componentes são conhecidos como **Componentes Visuais (VCL – Visual Component Library)**.

Os componentes visuais são **controles exibidos na interface gráfica**, que oferecem funcionalidades como **entrada, exibição e manipulação de dados**, seleção de opções e escolha de itens disponíveis. A inserção, recuperação e edição de informações tornam-se possíveis por meio da utilização desses componentes diretamente nos formulários da aplicação.

Além dos controles de entrada de dados, o **C++Builder** também fornece componentes destinados à **organização e estruturação do layout do formulário**, conhecidos como **contêineres (containers)**. A principal função de um contêiner é **agrupar componentes que possuem características ou finalidades em comum**, conforme a necessidade do desenvolvedor.

A utilização de contêineres contribui para uma interface mais organizada e intuitiva, facilitando a navegação e o entendimento do usuário ao trabalhar com os dados. A seguir, são apresentados os principais **componentes de entrada de dados disponíveis no C++Builder**.

EDIT

O componente **Edit** é um campo de texto destinado a receber ou exibir dados ao usuário. O conteúdo gerado por este componente pode ser utilizado por variáveis, funções, gravação de dados ou como complemento para outros componentes. Para adicionar um novo Edit ao formulário, é necessário abrir a paleta de componentes, em seguida expandir a categoria *Standard* e arrastar o ícone *TEdit* para o formulário. Ao ser adicionado, o componente vem com o nome de instância como conteúdo padrão, como *Edit1*, por exemplo.

As propriedades do Edit ajudam a definir algumas condições de comportamento do componente, muito utilizado para restrições de entrada, conforme segue abaixo:

Propriedade CharCase: define o modo de inserção das letras – normal (*ecNormal*), maiúsculo (*ecUpperCase*) ou minúsculo (*ecLowerCase*). Esta propriedade é útil quando se deseja, por exemplo, que o nome de um cliente seja obrigatoriamente digitado em maiúsculas, enquanto o seu e-mail deve ser somente em minúsculas.

Propriedade Color: define a cor de fundo do componente. Por padrão, o Delphi exibe uma lista de cores nativas do Sistema Operacional, mas novas cores podem ser exibidas ao clicar duas vezes sobre esta propriedade.

Propriedade NumbersOnly: nova nas versões mais recentes do C++Builder, esta propriedade define se o Edit aceitará somente números como entrada, quando marcada como *True*. Se esta for marcada como *False* (padrão), qualquer caractere será aceito.

Propriedade PasswordChar: quando o Edit for destinado a receber uma senha do usuário, esta propriedade é capaz de substituir os caracteres digitados por qualquer outro

caractere desejado. Por exemplo, se esta propriedade for preenchida com o caractere “#”, todo o conteúdo do Edit exibirá somente este caractere, ao invés do texto original inserido pelo usuário.

Propriedade *MaxLength*: define a quantidade máxima de caracteres que poderão ser inseridos no Edit. Esta propriedade é muito importante para evitar que o usuário digite uma quantidade maior de caracteres do que um campo no banco de dados pode aceitar, por exemplo.

Propriedade *ReadOnly*: quando marcada como *True*, o componente Edit não permitirá a inserção de dados pelo usuário, ou seja, o Edit será um controle somente para leitura de dados.

Propriedade *Text*: principal propriedade do componente Edit. Sua função é definir o conteúdo interno do componente. Quando o usuário digita um texto dentro do Edit, os dados poderão ser acessados através desta propriedade.

A maioria das propriedades do Edit pode ser acessada e alterada em tempo de execução da aplicação. Segue abaixo alguns exemplos de como trabalhar com as propriedades do Edit via código no C++Builder:

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    Edit1->Text = "Texto de Exemplo"; // altera o conteúdo do Edit
    Edit1->Color = clSkyBlue;        // Altera o fundo para a cor azul claro
    ShowMessage(Edit1->Text);        //Exibe uma mensagem com o conteúdo do Edit.
}
//-----
```

Código 1

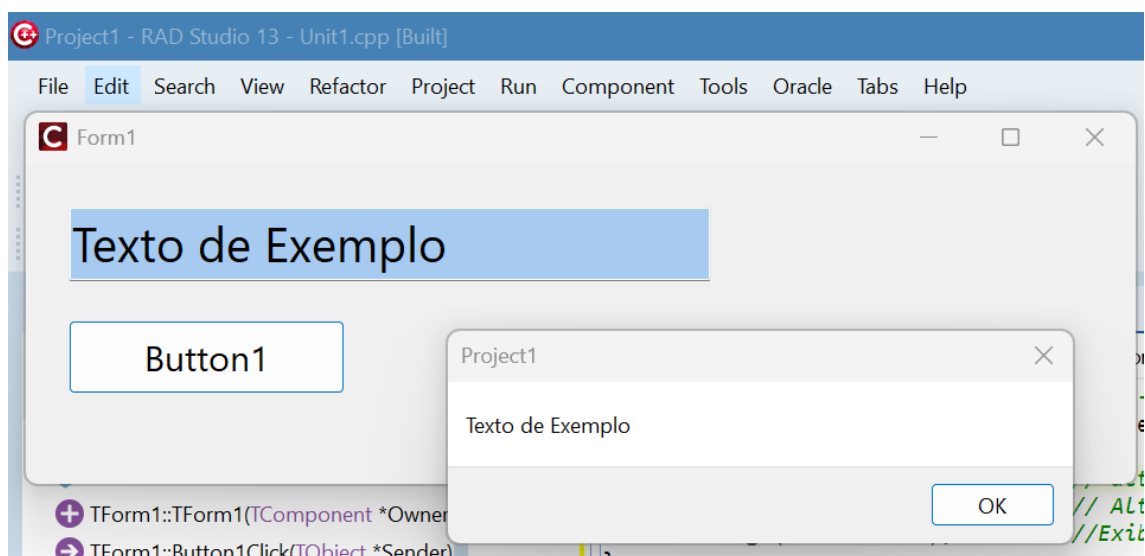


Figura 1: Trabalhando com o componente Edit

MASKEDIT

Em muitos casos é preciso personalizar um campo de texto de modo que ele obedeça a um formato de caracteres pré-definido para digitação. Este formato de

caracteres é conhecido no ambiente de programação como “máscara”. O componente MaskEdit é disponibilizado no C++Builder por ter esse diferencial quando comparado à Edit, ou seja, é possível formatar máscaras no campo de texto. Para isso, o componente MaskEdit conta com uma propriedade exclusiva chamada *EditMask*, responsável por configurar a máscara de entrada de dados. Note que esta propriedade possui um pequeno botão à esquerda com reticências. Ao clicar neste botão, o editor de máscaras é aberto em uma nova janela com alguns modelos de máscaras, além do campo *Input Mask* para personalização da máscara. Por exemplo, para adicionar uma máscara de um campo “Telefone”, basta digitar no campo *Input Mask* os caracteres “(99) 9999-9999”, conforme a Figura 2:

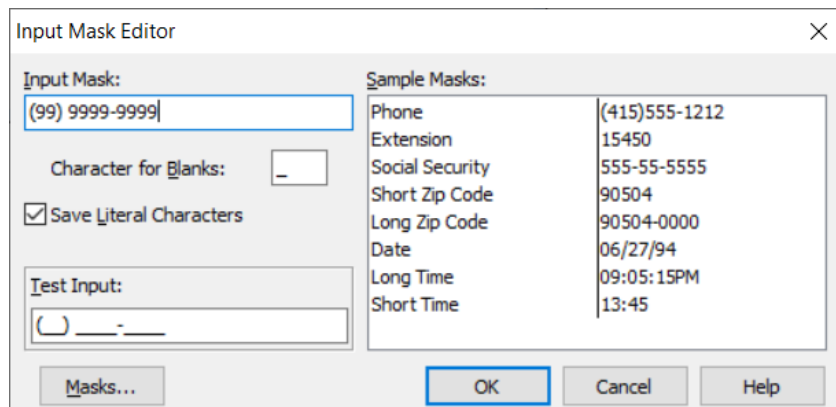


Figura 2: Editor de máscaras do componente MaskEdit

Desta forma, quando o usuário começar a digitar dentro deste campo, os parênteses e os hífens serão inseridos automaticamente à medida que os caracteres forem digitados. Vale ressaltar que os espaços aceitos na entrada de dados são definidos pelo número “9” no exemplo acima. Este número é utilizado pela propriedade *EditMask* para definir o caractere como obrigatório, conforme outros números e letras também aceitos por esta propriedade:

- 9 – entrada de número obrigatória
- 0 – entrada de número não obrigatória
- A – entrada de letra obrigatória
- a – entrada de letra não obrigatória

Ainda no editor de máscaras, a opção *Save Literal Characters*, quando marcada, permite que o texto do *MaskEdit* seja incluso com os caracteres de formatação (parênteses, hífens, pontos e outros caracteres especiais). Para que esta opção seja melhor compreendida, confira o exemplo abaixo:

Texto do MaskEdit: (22) 1234-5678

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    //Opção "Save Literal Characters" marcada
    ShowMessage (MaskEdit1->Text);
}
//-----
```

Código 2

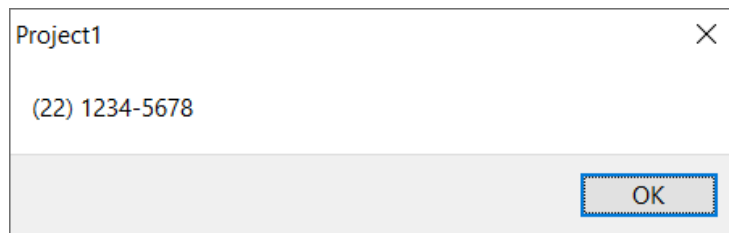


Figura 3: Exemplos com a opção “Save Literal Charaters” marcada

```

//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    //Opção "Save Literal Characters" desmarcada
    ShowMessage (MaskEdit1->Text);
}
//-----

```

Código 3

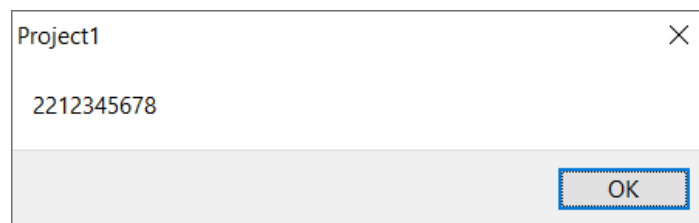


Figura 4: Exemplos com a opção “Save Literal Charaters” desmarcada

As demais propriedades do MaskEdit são semelhantes ao componente Edit discutido anteriormente.

DATETIMEPICKER

A entrada de valores de datas e horas em uma aplicação também é simplificada com a utilização do componente DateTimePicker. Ao adicioná-lo no formulário, é possível notar que o DateTimePicker possui um botão do lado direito do campo, responsável por exibir um pequeno calendário na tela quando clicado. A data selecionada no calendário é automaticamente atribuída ao campo DateTimePicker, permitindo que não seja necessário a digitação da data. As propriedades exclusivas do DateTimePicker são relacionadas à valores de datas e horas, conforme a seguir:

Propriedade CalAlignment: define a posição do calendário quando o botão é clicado – alinhado à esquerda do campo (*dtaLeft*) ou à direita (*dtaRight*).

Propriedade Date: define a data exibida no campo do DateTimePicker. Por padrão, o C++Builder atribui a data atual do sistema à essa propriedade, mas nada impede este valor seja alterado tanto em tempo de projeto quando no código-fonte em tempo de execução.

```

1 //-----
2 void __fastcall TForm1::Button1Click(TObject *Sender)
3 {
4     //Converte e atribui a data ao componente DateTimePicker1
20     DateTimePicker1->Date = StrToDate("08/12/1984");
5 }
6 //-----

```

Código 4: StrToDate é a conversão de String para Data.

Propriedade *DateFormat*: aqui é possível configurar a exibição da data no campo conforme a necessidade do desenvolvedor: Data curta (*dfShort* – somente números) ou Data Longa (*dfLong* – data por extenso).

Propriedade *DateMode*: quando configurada para *dmUpDown*, transforma o botão de calendário em dois botões com setas para cima e para baixo. Assim, o dia, mês ou ano podem ser incrementados ou decrementados em uma unidade cada vez que forem clicados. Se o valor desta propriedade for *dmComboBox*, o DateTimePicker volta a exibir o calendário como padrão.

Propriedade *Kind*: esta propriedade define o tipo de valor do campo DateTimePicker. A opção *dtkDate* permite a entrada e manipulação de datas no campo, enquanto a opção *dtkTime* é voltada para valores de horas.

Propriedades *MaxDate* / *MinDate*: definem a delimitação da maior e menor data permitida pelo campo. Estas propriedades são bastante úteis quando é necessário restringir a data selecionada pelo usuário. Ao configurar os valores de *MinDate* e *MaxDate*, o calendário exibirá somente os dias dentro deste intervalo de datas.

SPINEDIT

O componente SpinEdit possui a principal funcionalidade de incremento e decremento de valores numéricos. Para isso, o componente possui duas setas localizadas à direita no campo para tais finalidades. Apesar da simplicidade, o componente ainda conta com algumas propriedades úteis relacionadas ao seu comportamento:

Propriedade *Increment*: define o valor de incremento quando uma das setas é clicada no campo. Em alguns casos, por exemplo, é preciso que os valores sejam incrementados em 10 unidades ao invés de apenas 1.

Propriedade *Value*: tem basicamente a mesma função da propriedade *Text* do Edit, ou seja, é o valor do campo SpinEdit, e pode ser acessada em tempo de execução via código.

Propriedades *MinValue* / *MaxValue*: definem o menor e maior valor inteiro permitido pelo campo. Se o usuário tentar digitar um valor maior do que o especificado, o SpinEdit valida automaticamente a restrição e atribui o valor de *MaxValue* ao campo.

MEMO

Mesmo com a funcionalidade do campo Edit, muitas vezes é necessário digitar uma grande quantidade de informações, como observações, anotações ou outros dados que exigem várias linhas de texto. Para atender essa funcionalidade, o C++Builder disponibiliza o componente Memo. Quando adicionado ao formulário, o Memo já vem com um aspecto exclusivo para inserção de texto em várias linhas.

O diferencial do componente Memo, quando comparado aos outros componentes de texto, é o modo como o conteúdo é acessado via código. Note que o componente Memo não possui a propriedade *Text* para acesso ao conteúdo como o componente Edit. Para tal, o componente possui a propriedade *Lines*, que pode ser facilmente editada ao clicar no botão de reticências para abrir o editor de textos.

Portanto, para atribuir ou recuperar o conteúdo de um componente Memo via código, basta utilizar o comando *Lines.Text*, como segue o exemplo a seguir:

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    Memo1->Lines->Text = "Este é um campo de memorando. ";
    ShowMessage(Memo1->Lines->Text);
}
//-----
```

Código 5

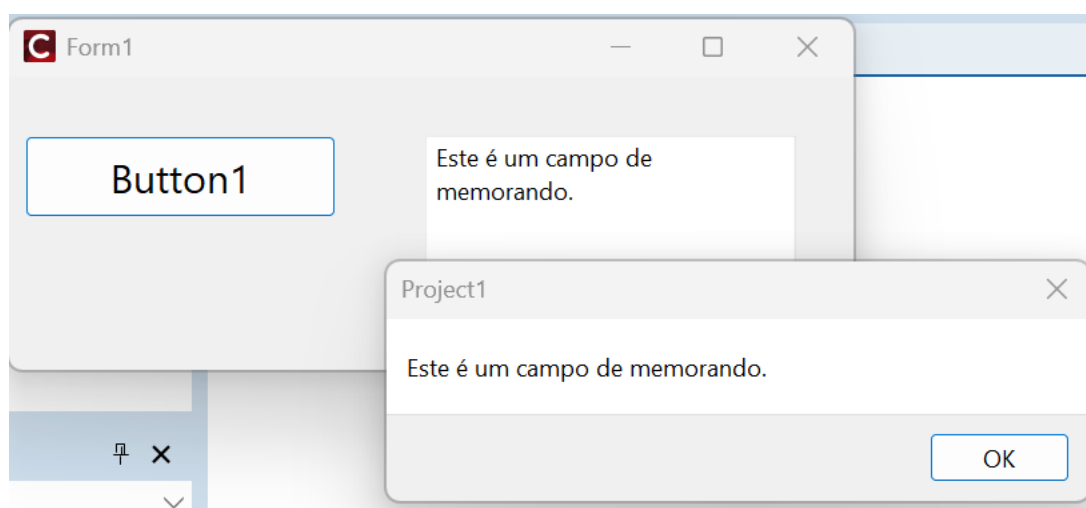


Figura 5: Exibindo uma mensagem com o conteúdo do componente Memo

Para adicionar quebras de linha em um Memo via código deve ser feita com `"\r\n"` ou usando constantes VCL.

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    Memo1->Lines->Text =
        "Este é um campo de memorando.\r\n"
        "Várias linhas podem ser adicionadas.";
    ShowMessage(Memo1->Lines->Text);
}
//-----
```

Código 6

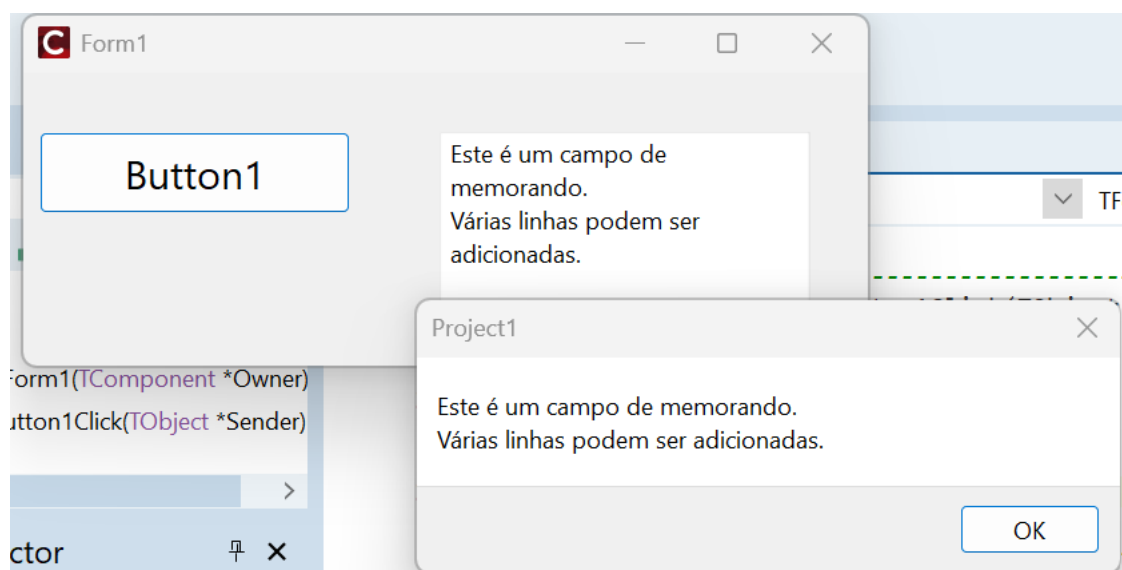


Figura 6: Exibindo uma mensagem com o conteúdo em duas linhas do componente Memo

Salvo estas observações sobre o conteúdo do Memo, o componente ainda traz consigo algumas propriedades exclusivas para alterar o comportamento do campo de texto:

Propriedade *ScrollBars*: configura a visibilidade das barras de rolagem do campo Memo quando o conteúdo excede as suas dimensões. A opção *ssNone* (padrão) não adiciona barras de rolagem, assim o campo Memo acompanha o texto conforme é digitado no campo. As opções *ssVertical* e *ssHorizontal* adicionam barras de rolagem vertical ou horizontal, respectivamente. Já a opção *ssBoth* adicionam as duas barras de rolagem ao mesmo tempo, tanto vertical quanto horizontal.

Propriedade *WantReturns*: quando habilitada, permite que o usuário tecle ENTER para adicionar uma quebra de linha no campo de texto. Se estiver desabilitada, as quebras de linha só serão adicionadas quando o conteúdo atingir o tamanho horizontal do campo Memo.

Propriedade *WantTabs*: tem a mesma característica da propriedade *WantReturns*, mas desta vez com a tecla TAB para adicionar tabulações no campo de texto. Por padrão, esta propriedade vem desabilitada, já que normalmente o TAB é utilizado para avançar o foco entre os campos.

Propriedade *WordWrap*: responsável por quebrar a linha quando o conteúdo atingir o tamanho horizontal do campo Memo. Se esta for desabilitada, o Memo acompanhará o texto conforme ele for digitado, ou seja, não haverá quebras de linha ao menos que o usuário pressione a tecla ENTER.

RICHEDIT

Em primeira instância, o componente RichEdit é bem parecido com o componente Memo, exceto por um grande diferencial: o componente RichEdit permite a formatação de textos além da gravação e leitura de arquivos de formato RTF (*Rich Text Format*). As Figuras 7 e 8 demonstram o modo como um texto selecionado no RichEdit pode ser formatado via código através do comando *SelAttributes*:

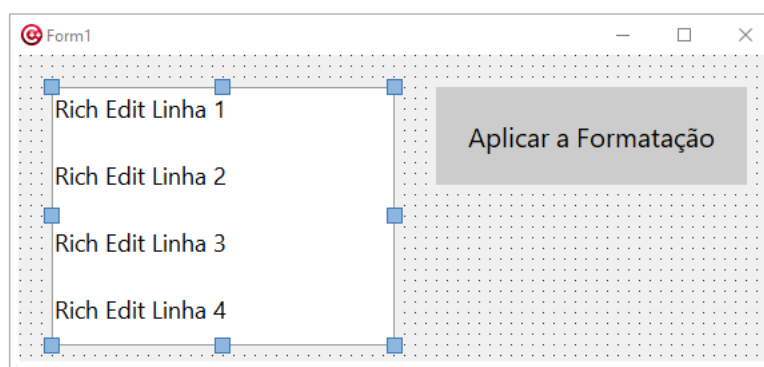


Figura 7: Trabalhando com o componente RichEdit

O código a seguir é implementado no botão “Aplicar Formatação”. Observe a forma como os valores são atribuídos às propriedades, ou seja, respeitando o tipo de dado que cada precisa receber. O estilo da fonte, por exemplo, deve ser especificado entre colchetes, enquanto o tamanho da fonte é do tipo inteiro e o nome da fonte está entre aspas por se tratar de uma string:

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    RichEdit1->SelAttributes->Style = TFontStyles() << fsBold; //aplica o estilo Negrito;
    RichEdit1->SelAttributes->Color = clGreen; //Aplica a cor verde;
    RichEdit1->SelAttributes->Size = 12; // Altera o tamanho do texto para 12
    RichEdit1->SelAttributes->Name = "Verdana"; //Aplica a fonte "Verdana"
}
//-----
```

Código 7

Após selecionar a linha 3 do RichEdit e clicar no botão “Aplicar Formatação”, surgirá o resultado:

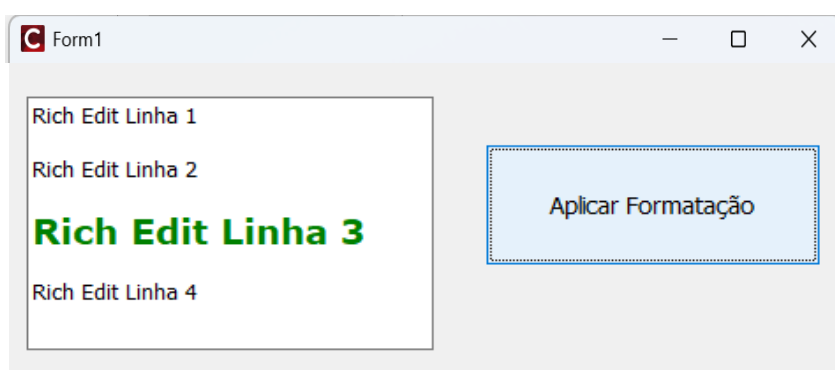


Figura 8: Resultado da formatação aplicada ao texto selecionado

A vantagem do RichEdit é a possibilidade de exportar o texto formatado para um arquivo RTF com apenas uma linha de comando, da mesma forma como um arquivo RTF também pode ser importado para um campo RichEdit:

```

//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    RichEdit1->Lines->SaveToFile('C:\\temp\\Arquivo.rtf'); //Salva o arquivo
    RichEdit1->Lines->LoadFromFile('C:\\temp\\Arquivo.rtf'); // Carrega o Arquivo
}
//-----

```

Código 8

COMBOBOX

O componente ComboBox pertence ao grupo de componentes para seleção de itens, geralmente encontrado em muitas aplicações. Também conhecido como Caixa de Seleção, um ComboBox é utilizado para armazenar um conjunto de itens disponíveis para que o usuário selecione uma opção. Para isso, o ComboBox conta com as seguintes propriedades:

Propriedade *Items*: responsável por definir os itens que estarão disponíveis para seleção. Ao clicar no botão de reticências, um editor é aberto para digitar os itens separados por quebras de linha. Os itens do ComboBox também podem ser adicionados em tempo de execução, de acordo com o exemplo abaixo:

```

//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    //Adiciona 3 itens ao combobox:
    ComboBox1->Items->Add("Brasil");
    ComboBox1->Items->Add("Estados Unidos");
    ComboBox1->Items->Add("Inglaterra");

    //Remove o item "Estados Unidos"
    ComboBox1->Items->Delete(1);
}
//-----

```

Código 9

Propriedade *ItemIndex*: acessa a opção selecionada pelo usuário. Cada item do ComboBox representa um índice, iniciando-se pelo número zero. Portanto, se o usuário selecionar o terceiro item do ComboBox, o *ItemIndex* deste item será o número 2.

Propriedade *AutoComplete*: quando habilitada, permite que o ComboBox complete a palavra conforme o usuário digitar no campo, caso esta palavra esteja disponível na propriedade *Items* do componente.

Propriedade *AutoDropDown*: permite que os itens do ComboBox sejam exibidos automaticamente quando o usuário digitar alguma tecla dentro do campo, sem a necessidade de clicar no botão de seta.

Propriedade *DropDownCount*: define a quantidade de itens visíveis quando o usuário clicar no botão de seta à direita da ComboBox. Por exemplo, se há 20 itens na Caixa de Seleção e a propriedade *DropDownCount* estiver configurada como 15, os últimos 5 itens não estarão visíveis ao menos que o usuário mova a barra de rolagem para baixo.

Propriedade *Sorted*: se estiver marcada, permite que o C++Builder ordene os valores inseridos na propriedade *Items* por ordem alfabética.

Propriedade Style: define o comportamento do ComboBox em relação à seleção de itens. A opção *csDropDown* (padrão) permite que o usuário digite qualquer texto independente se este esteja disponível na propriedade *Items*. Já as opções *csDropDownList* e *csOwnerDraw* obriga o usuário a selecionar um dos itens disponíveis na Caixa de Seleção. Por fim, a opção *csSimple* remove o botão de seta à direita e o ComboBox se iguala ao componente Edit, exceto pelo recurso de completar automaticamente as palavras.

CHECKBOX

O componente ComboBox visto anteriormente permite que o usuário selecione apenas uma opção entre os itens disponíveis. Caso seja necessário disponibilizar várias opções para seleção, o C++Builder disponibiliza o componente CheckBox. Este componente é um rótulo de texto acompanhado de uma caixa de verificação, onde o usuário “marca” ou “desmarca” a opção. A propriedade *Checked* é responsável por verificar se a caixa foi selecionada, como no exemplo abaixo:

```
//
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    if (CheckBox1->Checked) {
20      ShowMessage("A caixa esta selecionada.");
    }
    else
        ShowMessage("A caida não está selecionada.");
}
//
```

Código 10

Para definir o estado inicial do CheckBox, basta configurar a propriedade *State* para a opção desejada. Os estados *cbUnchecked* e *cbChecked* definem a caixa como desmarcada ou marcada, respectivamente, enquanto o estado *cbGrayed* define a caixa como “parcialmente” marcada. Este último estado pode ocorrer quando o CheckBox responde por um único conjunto de opções. Como exemplo na Figura 9, em um relatório de 5 páginas, apenas as páginas 1 e 2 estão marcadas para serem impressas, e há um CheckBox no formulário rotulado como “Imprimir todas as páginas”. Como 2 páginas estão marcadas e as outras 3 estão desmarcadas para imprimir, este checkBox estará em um estado *cbGrayed*, indicando que não há um valor em comum para todas as páginas. Vale ressaltar que, para que o estado *cbGrayed* esteja disponível, a propriedade *AllowGrayed* deve estar habilitada.

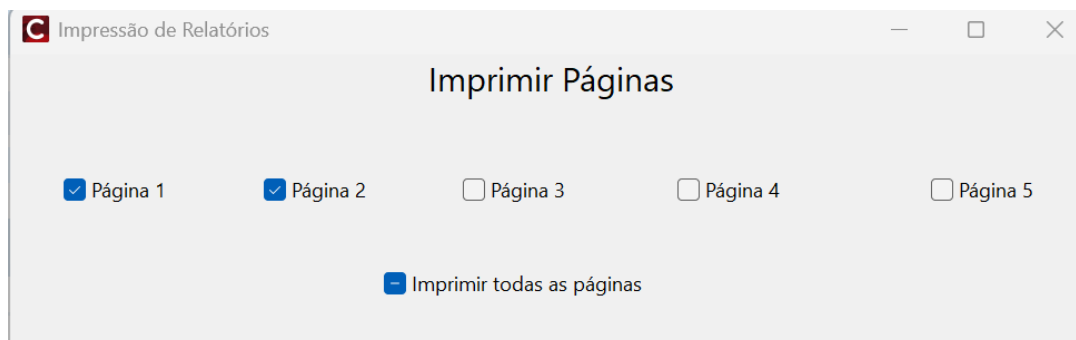


Figura 9: Tela de exemplo do estado *cbGrayed* do componente CheckBox

RADIOBUTTON

Este componente permite que apenas uma opção exclusiva seja selecionada dentro de um conjunto de opções, como o tipo de uma pessoa (Física ou Jurídica) ou o sexo (Masculino ou Feminino). Para verificar se a *RadioGroup* está selecionada, também é utilizada a propriedade *Checked*, assim como o componente *CheckBox*.

Para que o recurso de exclusividade seja funcional, os *RadioButtons* devem ser adicionados dentro de um mesmo contêiner, como um *Panel* ou um *GroupBox* (discutidos mais à frente).

RADIOGROUP

Em alguns casos pode ser necessário adicionar um grande número de opções em um mesmo contêiner (área do formulário composta por vários componentes), como por exemplo, selecionar apenas um título de livro entre 20 livros. Ao invés de adicionar 20 componentes do tipo *RadioButton*, o C++Builder disponibiliza um único componente capaz de armazenar todas as opções desejadas: o *RadioGroup*.

O componente *RadioGroup* é um contêiner de botões de seleção com um título para identificar o assunto das opções disponíveis. Para adicionar os botões, basta clicar nas reticências da propriedade *Items* para abrir um editor de texto. Cada linha neste editor representa um botão de opção no *RadioGroup*.

As opções do *RadioGroup* são interpretadas pelo C++Builder através de números, iniciando-se do número zero. Assim, para acessar a opção que foi selecionada pelo usuário, é preciso utilizar a propriedade *ItemIndex*, como no exemplo abaixo:

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    if (RadioGroup1->ItemIndex == 0)
    {
        ShowMessage("A primeira opção foi selecionada.");
    }
    else if (RadioGroup1->ItemIndex == 1)
    {
        ShowMessage("A segunda opção foi selecionada.");
    }
    else
    {
        ShowMessage("A terceira opção foi selecionada.");
    }
}
//-----
```

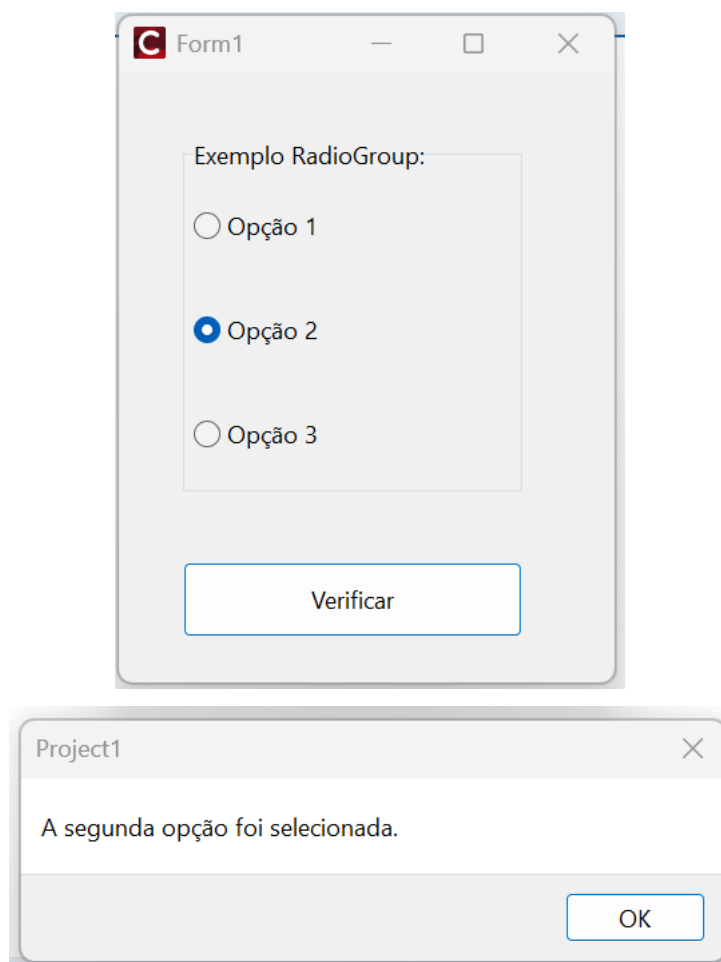
Código 11

Se houver um número maior de opções, é mais viável utilizar o comando **“switch”**:

```
//-----  
void __fastcall TForm1::Button1Click(TObject *Sender)  
{  
    switch (RadioGroup1->ItemIndex)  
    {  
        case 0:  
            ShowMessage("A primeira opção foi selecionada.");  
            break;  
        case 1:  
            ShowMessage("A segunda opção foi selecionada.");  
            break;  
        case 2:  
            ShowMessage("A terceira opção foi selecionada.");  
            break;  
    }  
}
```

Código 12

O resultado do código acima pode ser conferido na Figura 10:

**Figura 10: Trabalhando com o componente RadioGroup**

Note também que o componente `RadioGroup` é composto de um título para indicar o assunto das opções, que também pode ser alterado através da propriedade *Caption*.

PANEL

O componente `Panel` é utilizado como um contêiner para agrupar vários componentes com características ou assuntos em comum. No C++Builder, um contêiner pode ser compreendido como um espaço ou área dentro do formulário para organizar componentes ou delimitar o agrupamento de controles.

Para compreender a funcionalidade deste componente, basta arrastar alguns componentes para dentro do `Panel`, e então, alterar a posição do `Panel` no formulário selecionando-o e movendo com o mouse. Note que, ao mover o `Panel` pelo formulário, os componentes inseridos dentro dele também serão movidos na mesma escala. O componente `Panel` também contém algumas propriedades destinadas a alterar suas características visuais, como segue abaixo:

Propriedade *Caption*: define o texto interno do `Panel`, que por padrão é posicionado no centro do componente. O exemplo a seguir atribui a data e hora atual do sistema à propriedade *Caption* do `Panel`:

```
//-----  
void __fastcall TForm1::Button1Click(TObject *Sender)  
{  
    Panel1->Caption = FormatDateTime(  
        L"dd/mm/yyyy - hh:nn:ss",  
        Now()  
    );  
}  
//-----
```

Código 13: L" é usado pois a função `FormatDateTime` espera uma `UnicodeString`, não uma string ANSI. Não adicionar o L" pode dar erro de compilação.

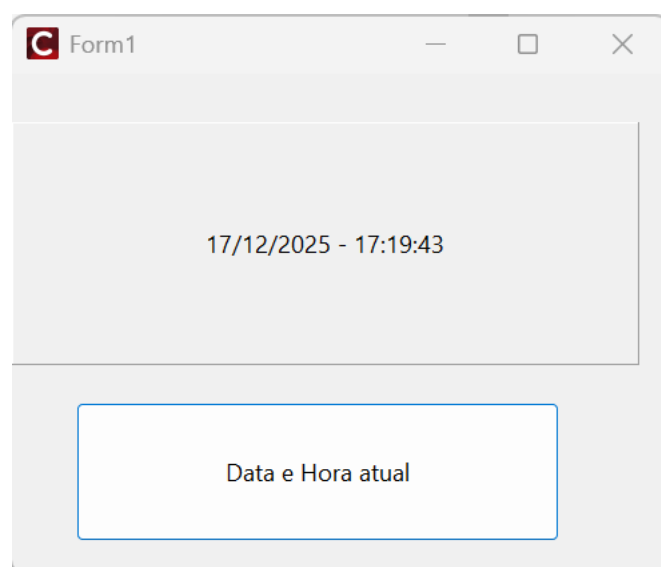


Figura 11: Exibindo a data e hora dentro do componente `Panel`

Propriedade *Alignment*: define o alinhamento horizontal do título (*Caption*) dentro do Panel. As opções *taLeftJustify* e *taRightJustify* alinham respectivamente o título à esquerda ou direita do componente. Por padrão, a opção *taCenter* vem selecionada para exibir o título no centro do Panel.

Propriedade *VerticalAlignment*: define o alinhamento vertical do título (*Caption*) dentro do Panel. A opção *taAlignBottom* exibe o título no rodapé do Panel, enquanto a opção *taAlignTop* o exibe no topo do componente. Já a opção *taVerticalCenter* exibe o título no centro como padrão.

Propriedade *ShowCaption*: quando desabilitada, o texto da propriedade *Caption* não é exibido no componente Panel.

Propriedades *BevelInner*, *BevelOuter* e *BevelKind*: responsáveis por alterar o estilo visual das bordas do Panel. Ao combinar os valores destas propriedades, é possível atribuir alguns efeitos visuais, como o alto relevo, obtido pela combinação do valor *bvRaised* nas propriedades *BevelInner* e *BevelOuter*.

GROUPBOX

Assim como o Panel, o componente GroupBox também tem a função de agrupar um conjunto de componentes como um contêiner. O diferencial do GroupBox é a forma como o título é exibido: sempre na parte superior à esquerda, acompanhando as bordas do componente. Por este motivo, o GroupBox não permite a alteração do visual das bordas, mas traz uma forma mais objetiva de exibir o título, dispensando a utilização de um componente Label, por exemplo.

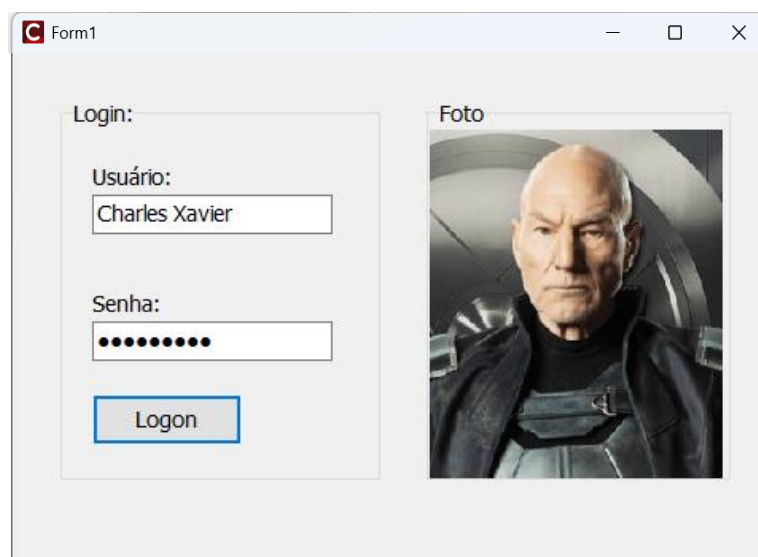


Figura 12: Exemplo de utilização do componente GroupBox

É importante ressaltar que ambos os componentes Panel e GroupBox são capazes de controlar a exclusividade de botões RadioButton. Portanto, ao adicionar dois ou mais RadioButtons dentro de um Panel ou de um GroupBox, apenas um RadioButton poderá ser selecionado pelo usuário.

BEVEL

Ao contrário dos componentes acima, o Bevel não deve ser utilizado como um contêiner de componentes. A principal função do componente Bevel é delimitar linhas, bordas e separadores visuais entre componentes, para auxiliar na usabilidade da aplicação. Apesar de sua aparência ser relativamente parecida com a de um Panel, o componente Bevel não possui propriedades de título (*Caption*) e alteração de bordas. Em contrapartida, o Bevel conta com outras propriedades visuais que ajudam o desenvolvedor a enriquecer o visual da aplicação:

Propriedade *Shape*: responsável pelo estilo visual do Bevel. As opções *bsBottomLine*, *bsLeftLine*, *bsRightLine* e *bsTopLine* transformam o Bevel em uma simples linha de separação, alinhada à opção desejada. A opção *bsBox* define o visual da Bevel como uma “caixa”, semelhante ao componente Panel, e por fim, a opção *bsSpacer* adiciona pontilhados às bordas do Bevel.

Propriedade *Style*: define se as bordas do Bevel serão “recuadas” (*bsLowered*) ou em relevo (*bsRaised*). Note que estas opções não surtem efeito caso o a propriedade *Shape* esteja configurada como *bsSpacer*.

Mesmo com as propriedades particulares que cada componente possui, é fundamental conhecer algumas propriedades que todos eles compartilham em comum, e que são importantes na confecção visual de uma aplicação.

Propriedade *TabOrder*: muitos usuários têm o hábito de pressionar a tecla TAB para avançar de campo ao invés de utilizar o mouse, e sem dúvida este hábito permite uma maior agilidade no preenchimento de dados. A propriedade *TabOrder* define a ordem de foco dos campos quando a tecla TAB é pressionada, iniciando-se pelo zero. Para que um campo receba o foco através do TAB, a propriedade *TabStop* deve estar habilitada. Da mesma forma, se o desenvolvedor desejar que determinado componente não receba foco, basta desabilitar a propriedade *TabStop*.

Propriedade *Align*: como o nome sugere, esta propriedade alinha o componente a uma determinada posição no formulário. A opção *alNone* vem selecionada como padrão, permitindo que o desenvolvedor posicione o controle manualmente no formulário. Para alinhar o componente ao topo deve-se selecionar *alTop*, enquanto *alBottom* alinha o componente no rodapé do formulário. Alinhamentos à esquerda e direita também são possíveis através das opções *alLeft* e *alRight*. Caso seja necessário que um componente cubra toda a dimensão do formulário, basta selecionar a opção *alClient*, conforme a Figura 13:

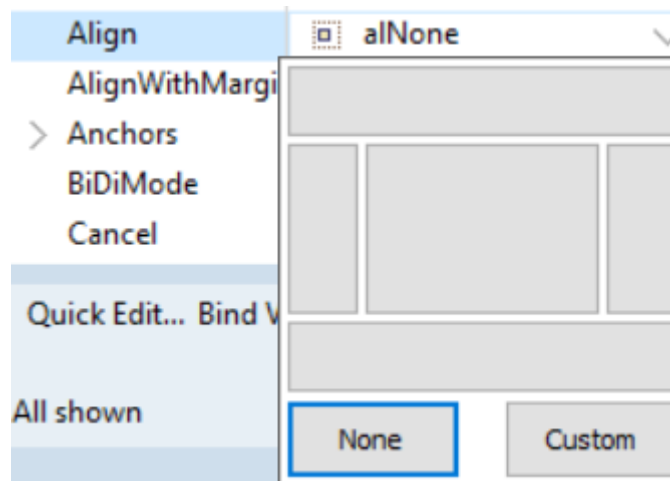


Figura 13: Opções de alinhamento através da propriedade *Align*

Propriedade *Ctl3D*: atribui um aspecto 3D ao visual dos componentes, comumente visto em aplicações mais modernas. Por padrão, o C++Builder mantém esta propriedade selecionada para cada componente que é adicionado ao formulário.

Propriedade *Cursor*: define o estilo do cursor quando este está sobre o componente. As opções disponíveis atendem aos tipos de cursores padrões do Sistema Operacional.

Propriedades *Left* e *Top*: definem a posição do componente dentro do contêiner ou formulário. A propriedade *Left* indica a distância do componente em relação à margem esquerda, e a propriedade *Top* em relação à margem superior. Estas propriedades são utilizadas para alinhar horizontalmente ou verticalmente um ou mais componentes no formulário.

Propriedades *Height* e *Width*: definem a altura e largura do componente em questão. Quanto maior o valor, maior será a dimensão do componente sobre o formulário. Estas propriedades, assim como *Left* e *Top*, podem ser simplesmente alteradas via código em tempo de execução.

Propriedade *Hint*: muitas vezes é necessário exibir uma dica para ajudar o usuário identificar a função do componente. Esta propriedade permite que o desenvolvedor especifique um texto que será exibido quando o usuário parar o cursor do mouse sobre o componente, como na Figura 14. Vale ressaltar que, para que a dica seja exibida, a propriedade *ShowHint* do componente deve estar marcada como *True*.

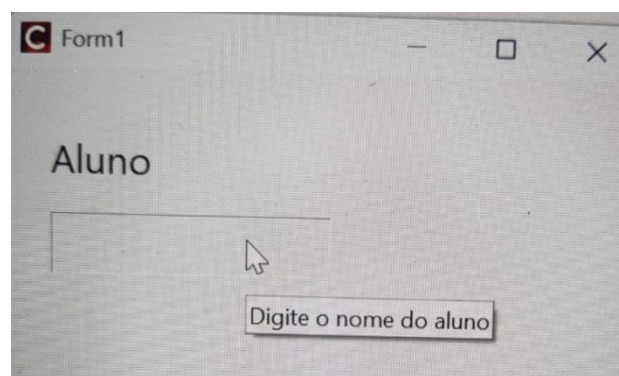


Figura 14: Exibindo uma dica do componente Edit ao usuário

Propriedade *Font*: responsável pelo estilo de fonte das letras digitadas no componente. Esta propriedade pode ser expandida ao clicar no símbolo “+” localizado à esquerda da palavra *Font*. Uma vez expandida, novas propriedades da fonte serão exibidas, como nome, tamanho, cor e estilo.

Propriedade *Name*: atribui um nome ao componente para facilitar a referência deste no código-fonte. É altamente recomendável renomear cada componente no formulário com um nome sugestivo, como “EditNome” ou “ComboBoxCidade”, por exemplo. Assim a implementação do código-fonte se torna mais fácil e legível.

```
//-----  
void __fastcall TForm1::Button1Click(TObject *Sender)  
{  
    EditNome->Text = "C++Builder";  
    ComboBoxCidade->ItemIndex = 2;  
}  
//-----
```

Código 14

Propriedade *Enabled*: define se o controle estará ativo no formulário. Quando estiver desabilitada, o controle não receberá foco e o seu valor não poderá ser alterado.

Propriedade *Visible*: define se o controle estará visível ao usuário quando a aplicação estiver em execução. Se esta propriedade for desmarcada, o controle somente estará visível em tempo de projeto para o desenvolvedor.